Tools

OpenClaw exposes **first-class agent tools** for browser, canvas, nodes, and cron. These tools replace the old `openclaw-*` Skills: tools are typed, don't require shell calls, and agents should depend on them directly.

Disabling Tools

You can globally allow/deny tools via `tools.allow` / `tools.deny` in `openclaw.json` (deny takes precedence). This blocks disallowed tools from being sent to the model provider.

```
{
  tools: { deny: ["browser"] },
}
```

Note:

- Matching is case-insensitive.
- Supports `*` wildcard (`"*"` for all tools).
- If `tools.allow` only references unknown or unloaded plugin tool names, OpenClaw will log a warning and ignore the allow list to ensure core tools remain available.

Tool Profiles (Base Allow Lists)

`tools.profile` sets a **base tool allow list** before `tools.allow` / `tools.deny`. Overridable per agent: `agents.list[].tools.profile`. Profiles:

- `minimal`: Only `session_status`
- `coding`: `group:fs`, `group:runtime`, `group:sessions`, `group:memory`, `image`
- `messaging`: `group:messaging`, `sessions_list`, `sessions_history`, `sessions_send`, `session_status`
- `full`: Unrestricted (same as unset)

Example (messaging by default, also allow Slack + Discord tools):

```
{
  tools: {
    profile: "messaging",
    allow: ["slack", "discord"],
  },
}
```

Example (coding profile, but deny exec/process everywhere):

```
{
  tools: {
    profile: "coding",
    deny: ["group:runtime"],
  },
}
```

Example (global coding profile, messaging-only support agent):

```
{
  tools: { profile: "coding" },
  agents: {
    list: [
      {
        id: "support",
        tools: { profile: "messaging", allow: ["slack"] },
      },
    ],
  },
}
```

Provider-Specific Tool Policies

Use `tools.byProvider` to **further restrict** tools for specific providers (or individual `provider/model`) without changing your global defaults. Overridable per agent: `agents.list[].tools.byProvider`. This applies after base tool profiles and before allow/deny lists, so it can only narrow the tool set. Provider keys accept `provider` (e.g., `google-antigravity`) or `provider/model` (e.g., `openai/gpt-5.2`). Example (keep global coding profile, but Google Antigravity uses minimal tools):

```
{
  tools: {
    profile: "coding",
    byProvider: {
      "google-antigravity": { profile: "minimal" },
    },
  },
}
```

Example (provider/model-specific allow list for unstable endpoints):

```
{
  tools: {
    allow: ["group:fs", "group:runtime", "sessions_list"],
    byProvider: {
      "openai/gpt-5.2": { allow: ["group:fs", "sessions_list"] },
    },
  },
}
```

Example (agent-specific override for a single provider):

```
{
  agents: {
    list: [
      {
        id: "support",
        tools: {
          byProvider: {
            "google-antigravity": { allow: ["message", "sessions_list"] },
          },
```

```
        },
      },
    ],
  },
}
```

Tool Groups (Shorthands)

Tool policies (global, agent, sandbox) support `group:*` entries, which expand to multiple tools. Use these in `tools.allow` / `tools.deny`. Available groups:

- `group:runtime`: `exec`, `bash`, `process`
- `group:fs`: `read`, `write`, `edit`, `apply_patch`
- `group:sessions`: `sessions_list`, `sessions_history`, `sessions_send`, `sessions_spawn`, `session_status`
- `group:memory`: `memory_search`, `memory_get`
- `group:web`: `web_search`, `web_fetch`
- `group:ui`: `browser`, `canvas`
- `group:automation`: `cron`, `gateway`
- `group:messaging`: `message`
- `group:nodes`: `nodes`
- `group:openclaw`: All built-in OpenClaw tools (excluding provider plugins)

Example (allow only file tools + browser):

```
{
  tools: {
    allow: ["group:fs", "browser"],
  },
}
```

Plugins + Tools

Plugins can register **additional tools** (and CLI commands) beyond the core set. See Plugins for installation + configuration, and Skills for how tool usage guidance is injected into prompts. Some plugins ship their own Skills with tools (e.g., voice-call plugin). Optional plugin tools:

- Lobster: Typed workflow runtime with resumable approvals (requires Lobster CLI on Gateway host).
- LLM Task: JSON-only LLM step for structured workflow output (optional schema validation).

Tool Inventory

`apply_patch`

Apply structured patches across one or more files. For multi-chunk edits. Experimental: enable via `tools.exec.applyPatch.enabled` (OpenAI models only).

`exec`

Run shell commands in the workspace. Core parameters:

- `command` (required)
- `yieldMs` (auto-background after timeout, default 10000)
- `background` (immediate background)
- `timeout` (seconds; kill if exceeded, default 1800)
- `elevated` (boolean; run on host if elevation mode enabled/allowed; only changes behavior when agent is sandboxed)
- `host` (`sandbox` | `gateway` | `node`)
- `security` (`deny` | `allowlist` | `full`)
- `ask` (`off` | `on-miss` | `always`)
- `node` (node id/name when `host=node`)
- Need a real TTY? Set `pty: true`.

Note:

- Returns `status: "running"` with `sessionId` when backgrounded.
- Use `process` to poll/log/write/kill/clear background sessions.
- If `process` is not allowed, `exec` runs synchronously and ignores `yieldMs` / `background`.
- `elevated` is gated by `tools.elevated` plus any `agents.list[].tools.elevated` overrides (both must allow), alias for `host=gateway` + `security=full`.
- `elevated` only changes behavior when agent is sandboxed (otherwise no-op).
- `host=node` can target macOS companion app or headless node hosts (`openclaw node run`).
- Gateway/node approvals and allowlists: Execution Approvals.

`process`

Manage background exec sessions. Core operations:

- `list`, `poll`, `log`, `write`, `kill`, `clear`, `remove`

Note:

- `poll` returns new output, exit status when complete.
- `log` supports line-based `offset` / `limit` (omit `offset` for last N lines).
- `process` is scoped per agent; sessions from other agents are not visible.

`web_search`

Search the web using Brave Search API. Core parameters:

- `query` (required)
- `count` (1-10; default from `tools.web.search.maxResults`)

Note:

- Requires Brave API key (recommended: `openclaw configure --section web`, or set `BRAVE_API_KEY`).
- Enable via `tools.web.search.enabled`.
- Responses are cached (default 15 minutes).
- See Web Tools for setup.

`web_fetch`

Fetch and extract readable content from URL (HTML → markdown/text). Core parameters:

- `url` (required)
- `extractMode` (`markdown` | `text`)
- `maxChars` (truncate long pages)

Note:

- Enable via `tools.web.fetch.enabled`.
- Responses are cached (default 15 minutes).
- For JS-heavy sites, prefer browser tool.
- See Web Tools for setup.
- See Firecrawl for optional anti-bot fallback.

`browser`

Control dedicated OpenClaw-managed browser. Core operations:

- `status`, `start`, `stop`, `tabs`, `open`, `focus`, `close`
- `snapshot` (aria/ai)
- `screenshot` (returns image block + `MEDIA:<path>`)
- `act` (UI actions: click/type/press/hover/drag/select/fill/resize/wait/evaluate)
- `navigate`, `console`, `pdf`, `upload`, `dialog`

Profile management:

- `profiles` — list all browser profiles with their status
- `create-profile` — create new profile with auto-assigned port (or `cdpUrl`)
- `delete-profile` — stop browser, delete user data, remove from config (local only)
- `reset-profile` — kill orphaned processes on profile port (local only)

Common parameters:

- `profile` (optional; defaults to `browser.defaultProfile`)
- `target` (`sandbox` | `host` | `node`)
- `node` (optional; pin specific node id/name)

Note:

- Requires `browser.enabled=true` (default `true`; set `false` to disable).
- All operations accept optional `profile` parameter for multi-instance support.
- When `profile` is omitted, uses `browser.defaultProfile` (default "chrome").
- Profile names: lowercase alphanumeric + hyphens only (max 64 chars).
- Port range: 18800-18899 (max ~100 profiles).
- Remote profiles support attach only (no start/stop/reset).
- If a browser-capable node is connected, tool may auto-route to it (unless you pin `target`).
- `snapshot` defaults to `ai` when Playwright installed; use `aria` for accessibility tree.
- `snapshot` also supports role snapshot options (`interactive`, `compact`, `depth`, `selector`), returning refs like `e12`.
- `act` needs `ref` from `snapshot` (number `12` in AI snapshot, or `e12` in role snapshot); use `evaluate` for rare CSS selector needs.
- Avoid `act` → `wait` by default; only use in special cases (no reliable UI state to wait for).
- `upload` can optionally pass `ref` to auto-click after ready.

- `upload` also supports `inputRef` (aria ref) or `element` (CSS selector) to set `<input type="file">` directly.

`canvas`

Drive Node Canvas (present, eval, snapshot, A2UI). Core operations:

- `present`, `hide`, `navigate`, `eval`
- `snapshot` (returns image block + `MEDIA:<path>`)
- `a2ui_push`, `a2ui_reset`

Note:

- Uses Gateway `node.invoke` under the hood.
- If `node` not provided, tool picks default (single connected node or local mac node).
- A2UI is v0.8 only (no `createSurface`); CLI will reject v0.9 JSONL with line error.
- Quick smoke test: `openclaw nodes canvas a2ui push --node <id> --text "Hello from A2UI"`.

`nodes`

Discover and locate paired nodes; send notifications; capture camera/screen. Core operations:

- `status`, `describe`
- `pending`, `approve`, `reject` (pairing)
- `notify` (macOS `system.notify`)
- `run` (macOS `system.run`)
- `camera_snap`, `camera_clip`, `screen_record`
- `location_get`

Note:

- Camera/screen commands require node app in foreground.
- Images return image block + `MEDIA:<path>`.
- Video returns `FILE:<path>` (mp4).
- Location returns JSON payload (lat/lon/accuracy/timestamp).
- `run` parameters: `command` argv array; optional `cwd`, `env` (`KEY=VAL`), `commandTimeoutMs`, `invokeTimeoutMs`, `needsScreenRecording`.

Example (`run`):

```
{
  "action": "run",
  "node": "office-mac",
  "command": ["echo", "Hello"],
  "env": ["FOO=bar"],
  "commandTimeoutMs": 12000,
  "invokeTimeoutMs": 45000,
  "needsScreenRecording": false
}
```

`image`

Analyze images using configured image model. Core parameters:

- `image` (required path or URL)
- `prompt` (optional; defaults to "Describe the image.")
- `model` (optional override)
- `maxBytesMb` (optional size limit)

Note:

- Available only when `agents.defaults.imageModel` is configured (primary or fallback), or when an implicit image model can be inferred from your default model + configured auth (best-effort pairing).
- Uses image model directly (independent of main chat model).

`message`

Send messages and channel operations across Discord/Google Chat/Slack/Telegram/WhatsApp/Signal/iMessage/MS Teams. Core operations:

- `send` (text + optional media; MS Teams also supports `card` for Adaptive Cards)
- `poll` (WhatsApp/Discord/MS Teams polls)
- `react` / `reactions` / `read` / `edit` / `delete`
- `pin` / `unpin` / `list-pins`
- `permissions`
- `thread-create` / `thread-list` / `thread-reply`
- `search`
- `sticker`
- `member-info` / `role-info`
- `emoji-list` / `emoji-upload` / `sticker-upload`
- `role-add` / `role-remove`
- `channel-info` / `channel-list`
- `voice-status`
- `event-list` / `event-create`
- `timeout` / `kick` / `ban`

Note:

- `send` routes WhatsApp via Gateway; other channels send directly.
- `poll` uses Gateway for WhatsApp and MS Teams; Discord polls send directly.
- When message tool call is bound to active chat session, sending is constrained to that session's targets to avoid cross-context leakage.

`cron`

Manage Gateway scheduled tasks and wakeups. Core operations:

- `status`, `list`
- `add`, `update`, `remove`, `run`, `runs`
- `wake` (enqueue system event + optional immediate heartbeat)

Note:

- `add` expects full cron object (same schema as `cron.add` RPC).

- `update` uses `{ id, patch }`.

`gateway`

Restart or apply updates to running Gateway process (in-place). Core operations:

- `restart` (authorize + send `SIGUSR1` for in-process restart; `openclaw gateway` in-place restart)
- `config.get` / `config.schema`
- `config.apply` (validate + write config + restart + wake)
- `config.patch` (merge partial update + restart + wake)
- `update.run` (run update + restart + wake)

Note:

- Use `delayMs` (default 2000) to avoid interrupting in-progress replies.
- `restart` disabled by default; enable with `commands.restart: true`.

`sessions_list` / `sessions_history` / `sessions_send` / `sessions_spawn` / `session_status`

List sessions, check transcript history, or send to another session. Core parameters:

- `sessions_list`: `kinds?`, `limit?`, `activeMinutes?`, `messageLimit?` (0 = none)
- `sessions_history`: `sessionKey` (or `sessionId`), `limit?`, `includeTools?`
- `sessions_send`: `sessionKey` (or `sessionId`), `message`, `timeoutSeconds?` (0 = fire-and-forget)
- `sessions_spawn`: `task`, `label?`, `agentId?`, `model?`, `runTimeoutSeconds?`, `cleanup?`
- `session_status`: `sessionKey?` (default current; accepts `sessionId`), `model?` (`default` clears override)

Note:

- `main` is canonical private chat key; global/unknown are hidden.
- `messageLimit > 0` gets last N messages per session (tool messages filtered).
- When `timeoutSeconds > 0`, `sessions_send` waits for final completion.
- Delivery/announcement happens after completion, best-effort; `status: "ok"` confirms agent run completed, not announcement delivered.
- `sessions_spawn` starts sub-agent run and sends announcement reply back to requester chat.
- `sessions_spawn` is non-blocking, returns `status: "accepted"` immediately.
- `sessions_send` runs reply ping-pong (reply `REPLY_SKIP` to stop; max rounds via `session.agentToAgent.maxPingPongTurns`, 0-5).
- After ping-pong, target agent runs an **announcement step**; reply `ANNOUNCE_SKIP` to suppress announcement.

`agents_list`

List agent ids that current session can target with `sessions_spawn`. Note:

- Results are limited by per-agent allowlists (`agents.list[].subagents.allowAgents`).
- When configured as `["*"]`, tool includes all configured agents and marks `allowAny: true`.

Parameters (Common)

Gateway-supported tools (`canvas`, `nodes`, `cron`):

- `gatewayUrl` (default `ws://127.0.0.1:18789`)
- `gatewayToken` (if auth enabled)
- `timeoutMs`

Browser tool:

- `profile` (optional; defaults to `browser.defaultProfile`)
- `target` (`sandbox` | `host` | `node`)
- `node` (optional; pin specific node id/name)

Recommended Agent Flows

Browser automation:

1. `browser` → `status` / `start`
2. `snapshot` (ai or aria)
3. `act` (click/type/press)
4. `screenshot` if you need visual confirmation

Canvas rendering:

1. `canvas` → `present`
2. `a2ui_push` (optional)
3. `snapshot`

Node targeting:

1. `nodes` → `status`
2. `describe` on selected node
3. `notify` / `run` / `camera_snap` / `screen_record`

Security

- Avoid direct `system.run`; only use `nodes` → `run` with explicit user consent.
- Respect user consent for camera/screen capture.
- Use `status/describe` before calling media commands to ensure permissions.

How Tools Are Presented to Agents

Tools are exposed through two parallel channels:

1. **System prompt text**: Human-readable list + guidance.
2. **Tool schemas**: Structured function definitions sent to model API.

This means agents see both "what tools exist" and "how to call them". If a tool doesn't appear in the system prompt or schema, the model cannot invoke it.

For more tool introduction content, refer to: [Tools - OpenClaw]